Rapport de conception ontologie

Projet de Modélisation de Connaissances et Web Sémantique

Auteur : Yasmina Farid

1. Objectif de modélisation

L'objectif de cette ontologie est de modéliser le domaine musical en représentant :

- les musiciens et les groupes,
- les chansons et les disques (albums, singles),
- les rôles musicaux,
- les récompenses et certifications,
- et surtout les relations temporelles entre musiciens et groupes.

en tenant compte du rôle occupé, de la période d'adhésion, et des changements possibles au cours du temps (un musicien qui rejoint un groupe, le quitte, puis y revient).

2. Utilisation de FOAF

FOAF sert de base pour les notions générales (personnes, agents, groupes) afin de garder un vocabulaire standard, lisible et réutilisable.

Classes FOAF réutilisées :

- `Musicien` ⊑ `foaf:Person` (individu humain)
- `Groupe` ⊑ `foaf:Group` (agent collectif)
- `aArtisteDisque : Disque → foaf:Agent`, `aInterprete : Chanson → foaf:Agent`, `laureat : PrixCeremonie → foaf:Agent` — `foaf:Agent` est utilisé chaque fois qu'une entité peut être indifféremment une personne ou un groupe (interprète solo ou en groupe, lauréat d'un prix).

Côté données : `foaf:firstName`, `foaf:family_name`, `foaf:gender` (fonctionnelle).

Classes FOAF volontairement évitées :

- `foaf:Organization` — aurait pu sembler adaptée aux groupes, mais introduit une dimension institutionnelle inutile pour le périmètre du projet.
- `foaf:Member` (lien direct musicien ↔ groupe) — ne permet pas d'exprimer *quand* la personne est membre, *quel rôle* elle occupe, ni de représenter des adhésions multiples et des allers-retours. C'est cette limite qui justifie la classe `AdhesionGroupe` (section suivante).

FOAF ne couvrant pas les besoins du sujet sur la nationalité, la temporalité et les rôles, ces notions sont ajoutées via `aNationalite`, `debut`/`fin` et `roleDansGroupe`.

3. Choix de conception des classes

3.1 Classes principales

- `Musicien` ⊑ `foaf:Person`
- `Groupe` ⊑ `foaf:Group`
- `Chanson` ⊑ `owl:Thing`
- `Disque` (abstraction commune), `Album` ⊑ `Disque`, `Single` ⊑ `Disque`

- `RoleMusical`
- `Recompense` (abstraction commune), spécialisée en `PrixCeremonie` ⊑ `Recompense` et `Certification` ⊑ `Recompense`
- `NiveauCertification`
- `Genre`

Ces classes couvrent les entités essentielles du domaine musical tout en restant suffisamment génériques pour être réutilisables.

3.2 Classe intermédiaire : `AdhesionGroupe` (choix clé)

`AdhesionGroupe` est le choix de modélisation central du projet. Elle représente une relation *n*-aire entre :

- un musicien,
- un groupe,
- un rôle musical,
- une période temporelle (`debut` / `fin`).

Sans cette classe intermédiaire, il ne serait pas possible d'associer un rôle à une appartenance, de dater une adhésion, ni de gérer les allers-retours d'un musicien dans un groupe. J'ai fait l'hypothèse qu'une adhésion à un groupe est un objet à part entière et non une simple relation binaire — c'est ce qui permet de représenter un départ suivi d'un retour comme deux instances distinctes d'`AdhesionGroupe` (par exemple 2008–2012 puis 2016–…), avec éventuellement un rôle différent entre les deux périodes.

Reconstruire la composition d'un groupe à une date t : un musicien est considéré membre à la date *t* si `debut` ≤ `t` et (`fin` absente ou `fin` ≥ `t`). Pour la composition associée à un disque, on peut associer le disque à une période puis retrouver les adhésions actives sur cette période.

3.3 « Valeurs » comme individus plutôt que littéraux

Certains concepts sont représentés comme des individus plutôt que des littéraux, car cela simplifie les requêtes et permet de définir des classes par restriction sur une valeur :

- **Genre** : individus (`Rock`, `Pop`, …) → permet `ChansonRock ≡ Chanson ⊓ (aGenre value Rock)`.
- **RoleMusical** : individus (`Chanteur`, `Guitariste`, …) — ensemble fini, facile à gérer sous forme de liste.
- **NiveauCertification** : individus (`Or`, `Platine`, `Diamant`, `2xDiamant`, …) → permet d'exprimer facilement des classes comme `DisqueDiamantFrance`.

4. Propriétés et structuration du modèle

4.1 Propriétés objet

Création musicale

- `aParolier`, `aCompositeur` : `Chanson` → `Musicien`
- `aInterprete` : `Chanson` → `foaf:Agent` (musicien ou groupe)

Groupes et rôles

- `aAdhesion` : `Groupe → AdhesionGroupe`
- `membre` : `AdhesionGroupe → Musicien`
- `groupe` : `AdhesionGroupe → Groupe`
- `roleDansGroupe` : `AdhesionGroupe → RoleMusical`

Disques et récompenses

- `dansDisque` : `Chanson → Disque`
- `aArtisteDisque` : `Disque → foaf:Agent`
- `aRecompense` : `Disque → Recompense`
- `laureat` : `PrixCeremonie → foaf:Agent`
- `niveauCertification` : `Certification → NiveauCertification`
- `aGenre` : `Chanson → Genre`

4.2 Propriétés de données

- identité du musicien : `foaf:firstName`, `foaf:family_name`, `aNationalite`
- temporalité : `debut`, `fin` (`xsd:gYear`, `fin` optionnelle si le membre est encore actif)
- prix : `nomPrix`, `categoriePrix`, `anneePrix`
- certifications : `anneeCertification`, `paysCertification` (fonctionnelle)

Hypothèse : les informations textuelles (nom du prix, catégorie) sont suffisantes pour le périmètre du projet.

5. Classes définies (inférées)

Ces classes servent à faire raisonner l'ontologie et à vérifier que la modélisation permet bien des inférences.

Chansons

- `ChansonRock ≡ Chanson ⊓ (aGenre value Rock)`
- `ChansonRockOuPop ≡ Chanson ⊓ (aGenre some {Pop, Rock})`
- `ChansonAvecPlusieursInterpretes ≡ Chanson ⊓ (aInterprete min 2 (Groupe ⊔ Musicien))`

Disques

- `DisqueAvecPlusieursMusiciens ≡ Disque ⊓ (aArtisteDisque min 2 Musicien)`
- `DisqueDiamantFrance ≡ Disque ⊓ (aRecompense some (Certification ⊓ niveauCertification ∈ {Diamant, 2xDiamant, 3xDiamant, 4xDiamant} ⊓ paysCertification = "France"))`

Récompenses

- `PrixVictoires ≡ PrixCeremonie ⊓ (nomPrix value "Victoires de la musique")`

Musiciens

- `JamaisSolo ≡ Musicien ⊓ inverse(aInterprete) some Chanson ⊓ inverse(aInterprete) only ChansonAvecPlusieursInterpretes` — `ChansonAvecPlusieursInterpretes` sert ici de classe intermédiaire, et `some Chanson` évite que la classe soit vraie « par défaut » pour un musicien sans chansons.

L'ontologie finale (`Chansons.owl`) contient également d'autres classes inférées répondant aux demandes du sujet (`GroupeMixte`, `SoloSeulement`, `DansPlusieursGroupes`, `AuteurInterprete`, `CompositeurRockSeulement`), non détaillées ici par souci de concision.

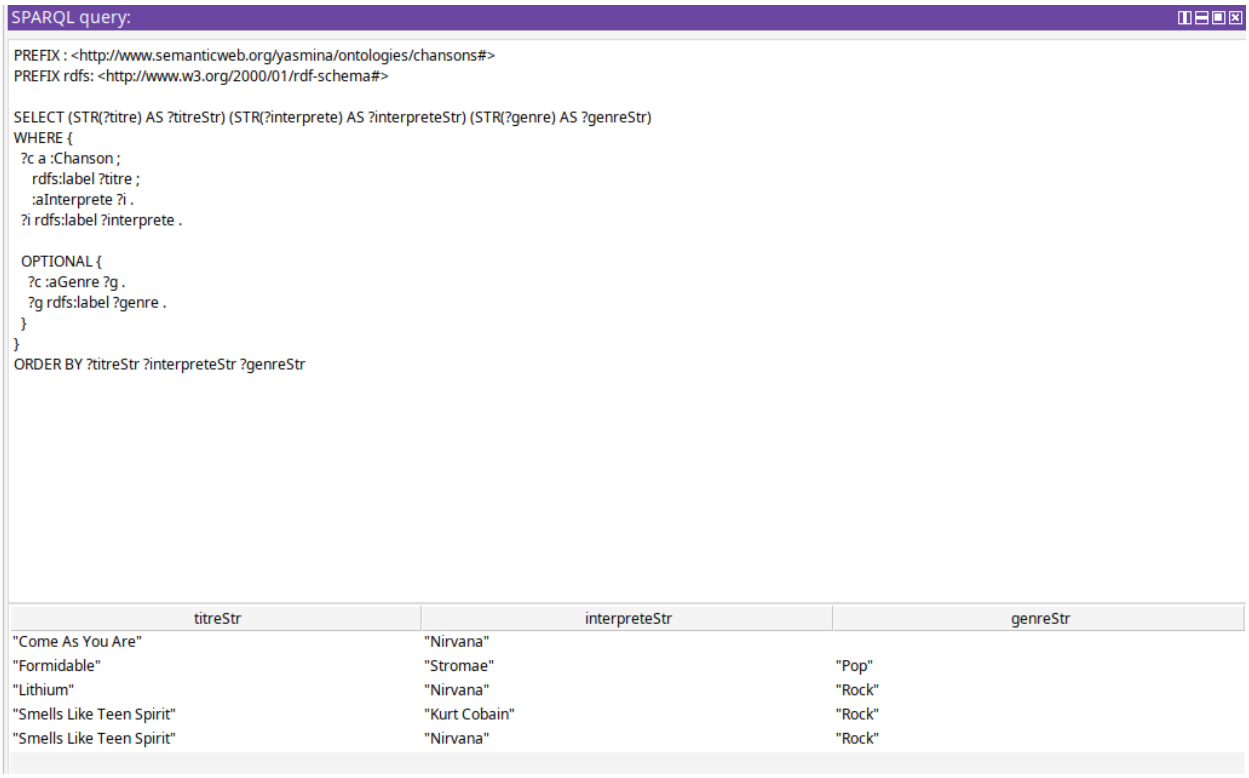
6. Résultats des requêtes de validation

Le modèle a été testé avec les 8 requêtes SPARQL et les 2 requêtes SQWRL demandées par le sujet dans Protégé (moteur Pellet/HermiT selon l'onglet). Toutes ont abouti à un résultat correct, **à l'exception de la dernière (SQWRL n°10)**, qui a échoué à cause de contraintes d'usage du langage (limitation de SQWRL sur ce type d'agrégation groupée) que je n'ai pas réussi à débugger dans le temps imparti.

Préfixe de base pour toutes les requêtes SPARQL : `PREFIX : <http://www.semanticweb.org/yasmina/ontologies/chansons#>`

SPARQL-01 — Titres, interprète et genre (genre optionnel)

```
SELECT (STR(?titre) AS ?titreStr) (STR(?interprete) AS ?interpreteStr) (STR(?genre) AS ?genreStr)
WHERE {
  ?c a :Chanson ; rdfs:label ?titre ; :aInterprete ?i .
  ?i rdfs:label ?interprete .
  OPTIONAL { ?c :aGenre ?g . ?g rdfs:label ?genre . }
}
ORDER BY ?titreStr ?interpreteStr ?genreStr
```

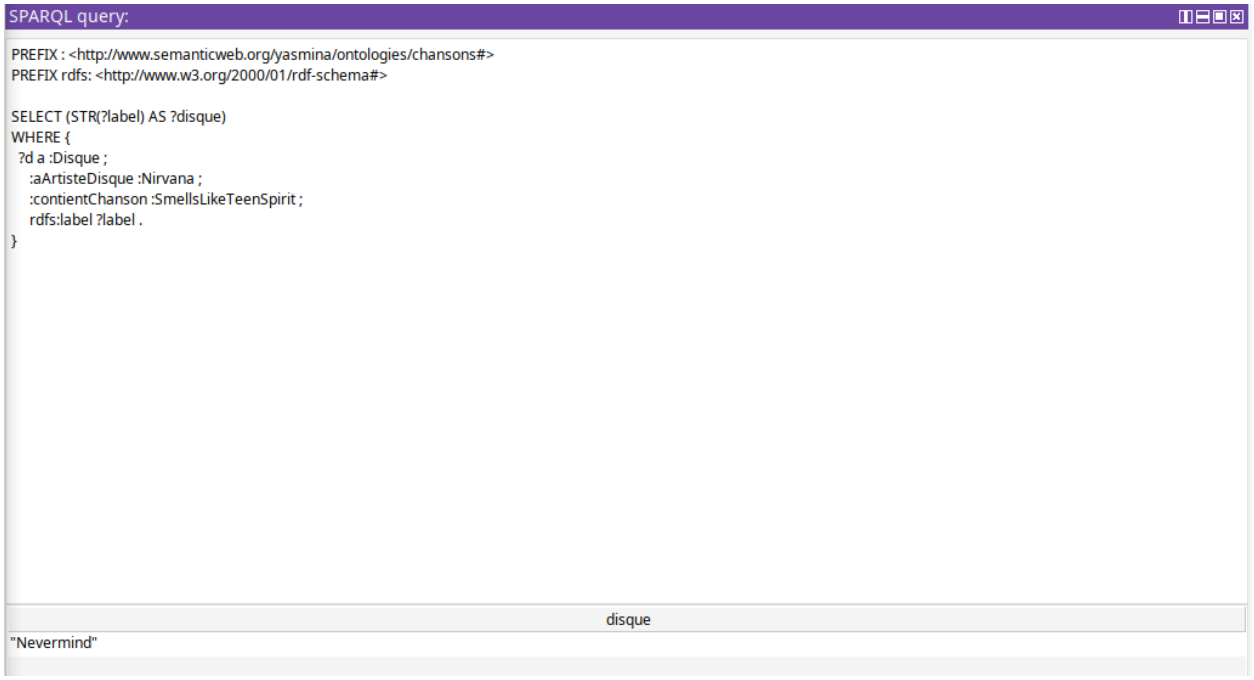


Résultat SPARQL-01

SPARQL-02 — Disque de Nirvana contenant Smells Like Teen Spirit

```
SELECT (STR(?label) AS ?disque)
WHERE {
  ?d a :Disque ; :aArtisteDisque :Nirvana ; :contientChanson :SmellsLikeTeenSpirit ; rdfs:label ?label
}
```

Résultat : **"Nevermind"**.

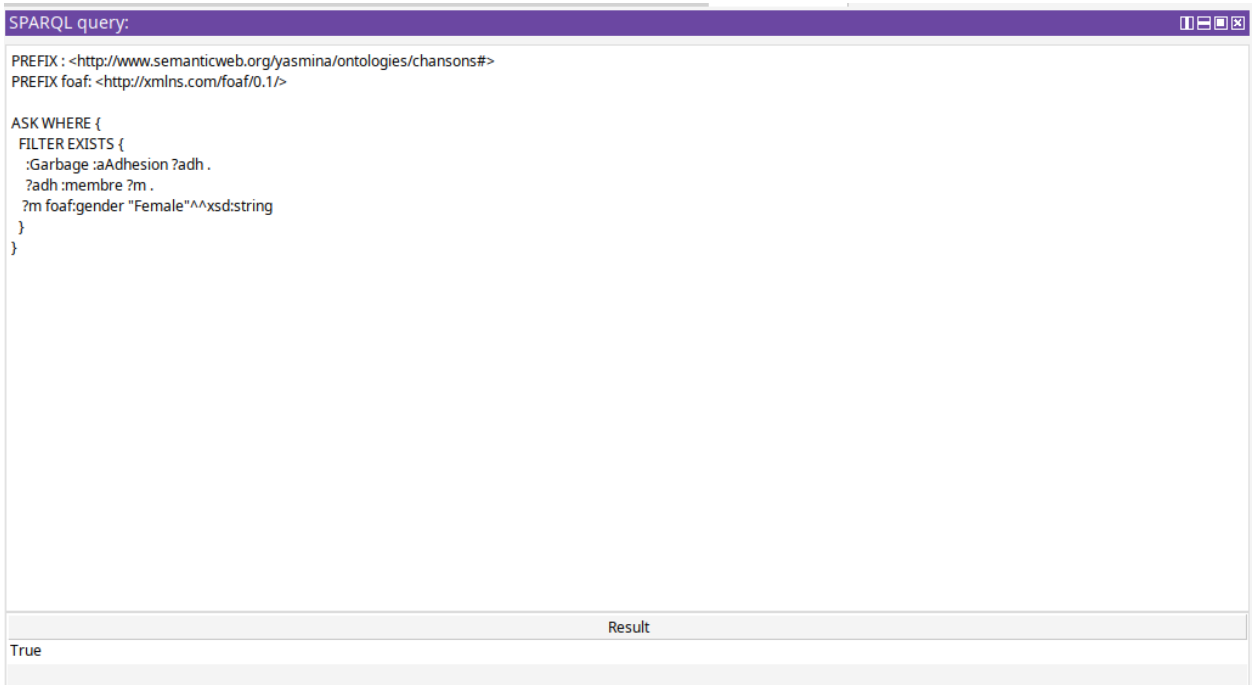


Résultat SPARQL-02

SPARQL-03 — ASK : Garbage a-t-il au moins un membre féminin ?

```
ASK WHERE {
  FILTER EXISTS {
    :Garbage :aAdhesion ?adh . ?adh :membre ?m . ?m foaf:gender ?g . FILTER(?g = "Female")
  }
}
```

Résultat : **True**.

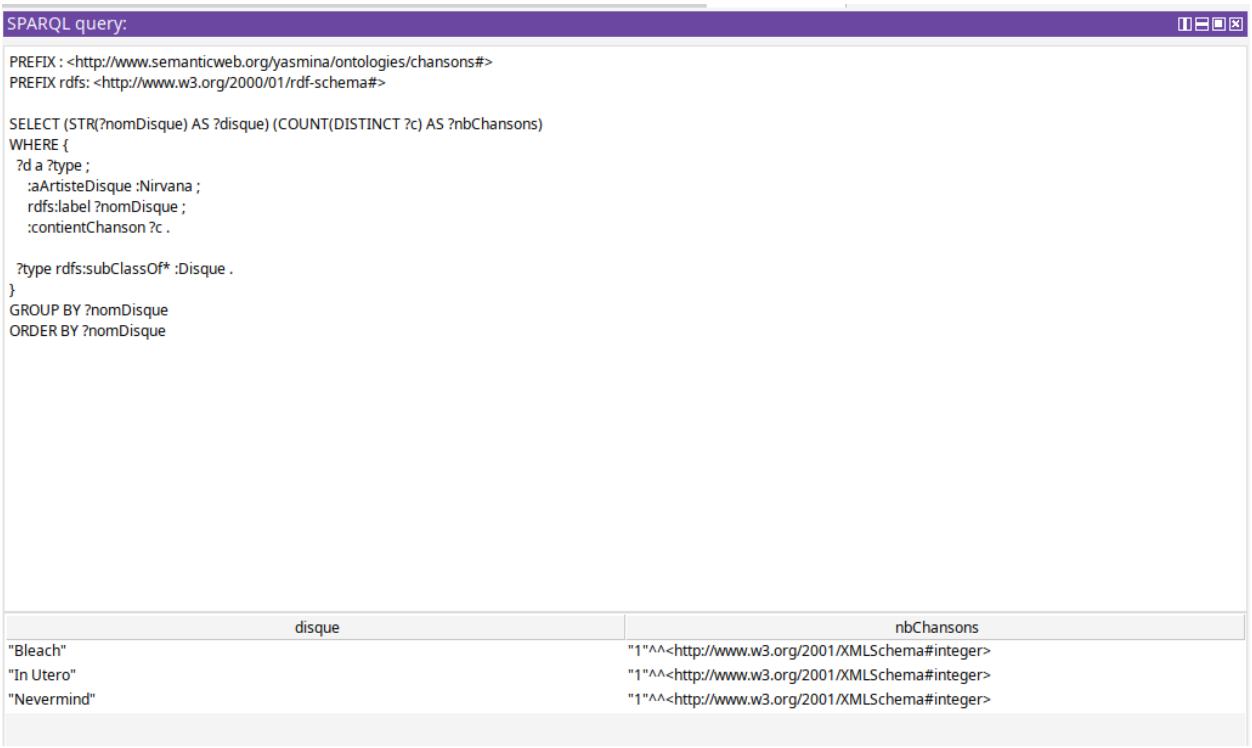


Résultat SPARQL-03

SPARQL-04 — Nombre de chansons par disque de Nirvana (Album + Single inclus)

```
SELECT (STR(?nomDisque) AS ?disque) (COUNT(DISTINCT ?c) AS ?nbChansons)
WHERE {
  ?d a ?type ; :aArtisteDisque :Nirvana ; rdfs:label ?nomDisque ; :contientChanson ?c .
  ?type rdfs:subClassOf* :Disque .
}
GROUP BY ?nomDisque ORDER BY ?nomDisque
```

Résultat : Bleach = 1, In Utero = 1, Nevermind = 1 (jeu de données de test avec une chanson représentative par disque).

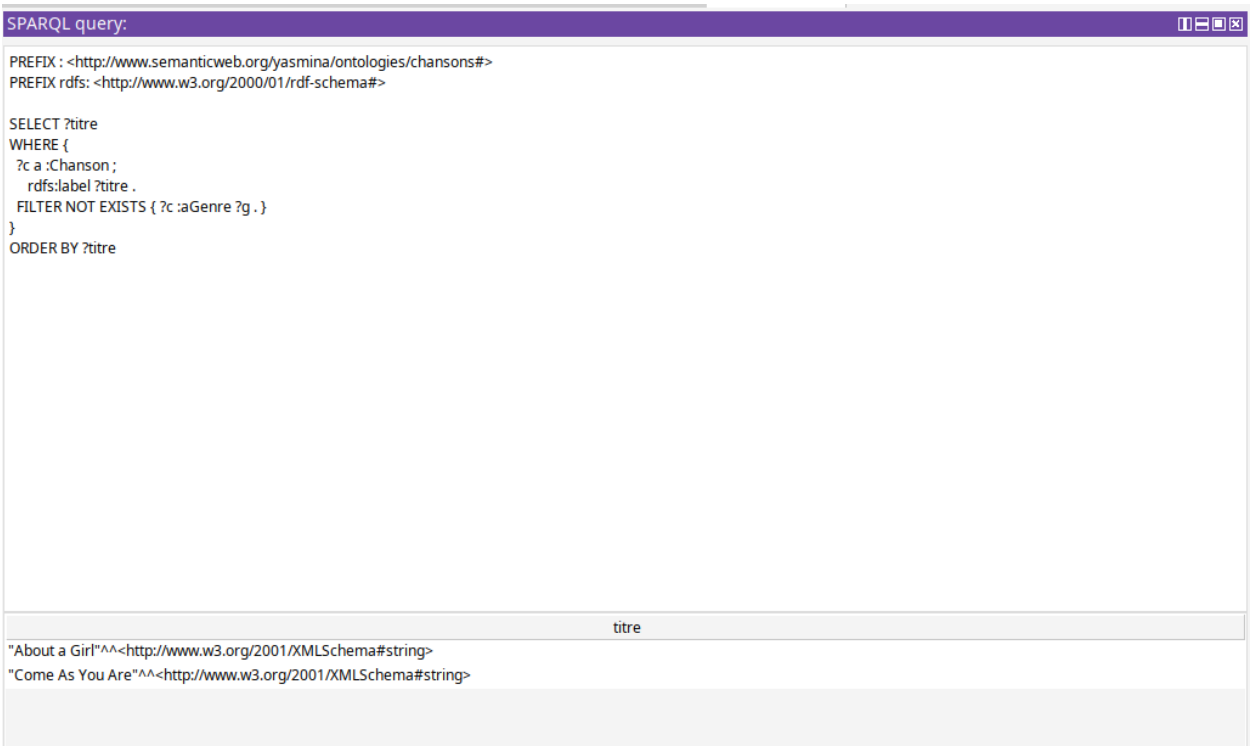


Résultat SPARQL-04

SPARQL-05 — Chansons sans genre renseigné

```
SELECT ?titre
WHERE {
  ?c a :Chanson ; rdfs:label ?titre .
  FILTER NOT EXISTS { ?c :aGenre ?g . }
}
ORDER BY ?titre
```

Résultat : "About a Girl", "Come As You Are" — utile pour repérer les données incomplètes.



Résultat SPARQL-05

SPARQL-06 — Récompenses (Prix ou Certification) obtenues en 2014 ou 2015

```
SELECT (STR(?nomArtiste) AS ?artisteNom) (STR(?nomRec) AS ?recompenseNom) (STR(?annee) AS ?anneeStr)
WHERE {
  ?d :aArtisteDisque ?artiste ; :aRecompense ?rec .
  ?artiste rdfs:label ?nomArtiste . ?rec rdfs:label ?nomRec .
  { ?rec a :PrixCeremonie ; :anneePrix ?annee . }
  UNION
  { ?rec a :Certification ; :anneeCertification ?annee . }
  FILTER(?annee = "2014"^^xsd:gYear || ?annee = "2015"^^xsd:gYear)
}
ORDER BY ?annee ?nomArtiste ?nomRec
```

Résultat : Stromae — Victoires de la musique 2014 ; Alain Souchon — Victoires de la musique 2015 ; Laurent Voulzy — Victoires de la musique 2015.

SPARQL query:

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT (STR(?nomArtiste) AS ?artisteNom)
     (STR(?nomRec) AS ?recompenseNom)
     (STR(?annee) AS ?anneeStr)
WHERE {
 ?d :aArtisteDisque ?artiste ;
   :aRecompense ?rec .

 ?artiste rdfs:label ?nomArtiste .
 ?rec rdfs:label ?nomRec .

 {
  ?rec a :PrixCeremonie ;
      :anneePrix ?annee .
 }
 UNION
 {
  ?rec a :Certification ;
      :anneeCertification ?annee .
 }

 FILTER(?annee = "2014"^^xsd:gYear || ?annee = "2015"^^xsd:gYear)
}
ORDER BY ?annee ?nomArtiste ?nomRec
```

artisteNom	recompenseNom	anneeStr
"Stromae"	"Victoires de la musique 2014"	"2014"
"Alain Souchon"	"Victoires de la musique 2015"	"2015"
"Laurent Voulzy"	"Victoires de la musique 2015"	"2015"

Résultat SPARQL-06

SPARQL-07 — Groupes ayant moins de 4 disques

```
SELECT (STR(?nomGroupe) AS ?groupe) (COUNT(DISTINCT ?d) AS ?nbDisques)
WHERE {
  ?d :aArtisteDisque ?g . ?g a :Groupe ; rdfs:label ?nomGroupe .
  ?d a ?typeDisque . ?typeDisque rdfs:subClassOf* :Disque .
}
GROUP BY ?nomGroupe HAVING (COUNT(DISTINCT ?d) < 4) ORDER BY ?nbDisques ?nomGroupe
```

Résultat : Nirvana — 3 disques.

SPARQL query:

```
PREFIX : <http://www.semanticweb.org/yasmina/ontologies/chansons#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT (STR(?nomGroupe) AS ?groupe)
     (COUNT(DISTINCT ?d) AS ?nbDisques)
WHERE {
 ?d :aArtisteDisque ?g .
 ?g a :Groupe ;
   rdfs:label ?nomGroupe .

 # pour inclure Album + Single (sous-classes de Disque)
 ?d a ?typeDisque .
 ?typeDisque rdfs:subClassOf* :Disque .
}
GROUP BY ?nomGroupe
HAVING (COUNT(DISTINCT ?d) < 4)
ORDER BY ?nbDisques ?nomGroupe
```

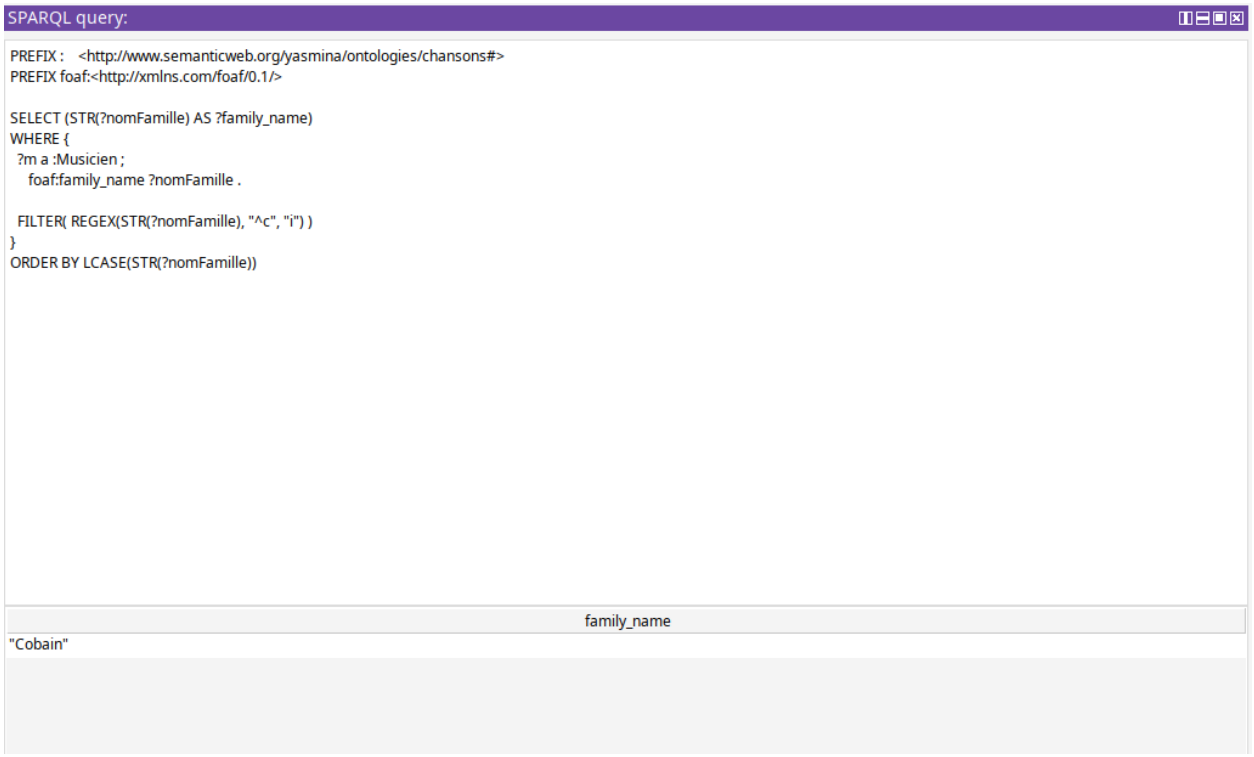
groupe	nbDisques
"Nirvana"	"3"^^<http://www.w3.org/2001/XMLSchema#integer>

Résultat SPARQL-07

SPARQL-08 — Musiciens dont le nom de famille commence par "c" (regex, insensible à la casse)

```
SELECT (STR(?nomFamille) AS ?family_name)
WHERE {
  ?m a :Musicien ; foaf:family_name ?nomFamille .
  FILTER( REGEX(STR(?nomFamille), "^c", "i") )
}
ORDER BY LCASE(STR(?nomFamille))
```

Résultat : "Cobain".

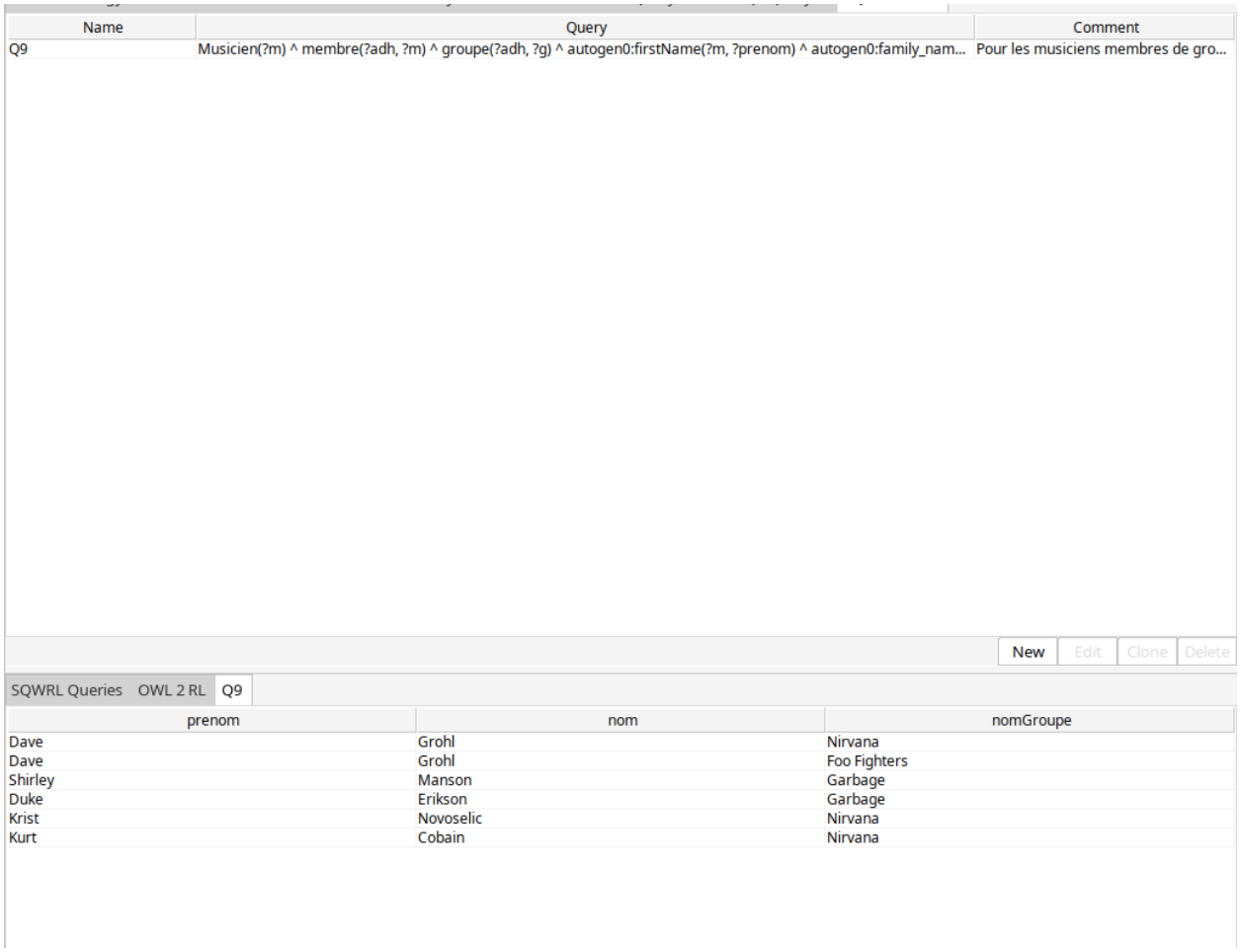


Résultat SPARQL-08

SQWRL-09 — Prénom, nom et groupe via les adhésions

```
Musicien(?m) ^ membre(?adh, ?m) ^ groupe(?adh, ?g) ^
autogen0:firstName(?m, ?prenom) ^ autogen0:family_name(?m, ?nom) ^ autogen0:name(?g, ?nomGroupe)
-> sqwrl:select(?prenom, ?nom, ?nomGroupe)
```

Résultat : Dave Grohl (Nirvana), Dave Grohl (Foo Fighters), Shirley Manson (Garbage), Duke Erikson (Garbage), Krist Novoselic (Nirvana), Kurt Cobain (Nirvana) — la requête illustre bien la gestion des adhésions multiples d'un même musicien à des groupes différents (Dave Grohl).



Name	Query	Comment
Q9	Musicien(?m) ^ membre(?adh, ?m) ^ groupe(?adh, ?g) ^ autogen0:firstName(?m, ?prenom) ^ autogen0:family_nam...	Pour les musiciens membres de gro...

New Edit Clone Delete

SQWRL Queries OWL 2 RL Q9

prenom	nom	nomGroupe
Dave	Grohl	Nirvana
Dave	Grohl	Foo Fighters
Shirley	Manson	Garbage
Duke	Erikson	Garbage
Krist	Novoselic	Nirvana
Kurt	Cobain	Nirvana

Résultat SQWRL-09

SQWRL-10 — Compter les adhésions "Female" par groupe (tentative, échec)

```
AdhesionGroupe(?adh) ^ membre(?adh, ?m) ^ groupe(?adh, ?g) ^
autogen0:gender(?m, "Female") ^ autogen0:name(?g, ?nomGroupe)
-> sqwrl:select(?nomGroupe) ^ sqwrl:count(?adh) ^ sqwrl:groupBy(?nomGroupe) ^ sqwrl:orderBy(?nomGroupe)
```

Cette requête n'a pas produit de résultat exploitable dans Protégé (contrainte d'usage de `sqwrl:count`/`sqwrl:groupBy` combinés que je n'ai pas réussi à contourner) ; aucune capture n'est donc fournie pour celle-ci.

7. Limites et améliorations possibles

- La requête SQWRL-10 (comptage groupé des adhésions féminines par groupe) n'a jamais abouti — une piste non explorée serait de reformuler le comptage en SPARQL (`GROUP BY`/`COUNT`) plutôt qu'en SQWRL, ce langage étant plus limité sur les agrégations.
- Le jeu de données de test reste petit (quelques musiciens, groupes et disques), suffisant pour valider les choix de modélisation mais pas pour représenter un catalogue réaliste.
- Les données prix/certifications sont des littéraux textuels simples (`nomPrix`, `categoriePrix`) plutôt que des individus structurés, ce qui limite les requêtes possibles sur ces catégories (choix assumé, documenté en section 4.2).

8. Contributrice

Ce projet a été réalisé par **Yasmina Farid** dans le cadre du cours de Modélisation de Connaissances et Web Sémantique (M1). La conception de l'ontologie (choix des classes, de la classe intermédiaire `AdhesionGroupe`, des propriétés, des classes inférées) ainsi que l'écriture et la validation de l'ensemble des requêtes SPARQL et SQWRL ont été réalisées seule, comme en attestent les notes de conception (`log_conception`) et les brouillons de travail conservés dans ce dépôt.